

Algorithms in C

A Practical Reference

Fourteen classic algorithms implemented in C — sorting, graph traversal, shortest paths, minimum spanning trees, string matching, dynamic programming, backtracking and branch & bound — with clean, consistently formatted, syntax highlighted source code for study and lab reference.

01 Merge Sort

02 Quick Sort

03 Breadth First Search

04 Depth First Search

05 Topological Sort

06 Heap Sort

07 Floyd–Warshall Algorithm

08 Dijkstra's Algorithm

09 Kruskal's Algorithm

10 Prim's Algorithm

11 0/1 Knapsack

12 N–Queens

13 Horspool's Algorithm

14 Job Assignment Problem

CONTENTS

Table of Contents

01	● Merge Sort	Sorting	3
02	● Quick Sort	Sorting	5
03	● Breadth First Search	Graph Traversal	8
04	● Depth First Search	Graph Traversal	10
05	● Topological Sort	Graph Ordering	11
06	● Heap Sort	Sorting	13
07	● Floyd–Warshall Algorithm	Shortest Path	16
08	● Dijkstra's Algorithm	Shortest Path	17
09	● Kruskal's Algorithm	Minimum Spanning Tree	19
10	● Prim's Algorithm	Minimum Spanning Tree	21
11	● 0/1 Knapsack	Dynamic Programming	23
12	● N–Queens	Backtracking	25
13	● Horspool's Algorithm	String Matching	27
14	● Job Assignment Problem	Branch & Bound	29

01

SORTING · DIVIDE & CONQUER

Merge Sort

TIME COMPLEXITY

$O(n \log n)$ — best, average and worst case

SPACE COMPLEXITY

$O(n)$ auxiliary array for merging

STABLE

Yes

Key idea: Split the array into two halves, sort each half recursively, then **merge** the two sorted halves back together by repeatedly picking the smaller of the two front elements. Because the array is always split exactly in half, the recursion depth is $\log_2(n)$ and merging at every level costs $O(n)$, giving the guaranteed $O(n \log n)$ bound regardless of the input order.

01_merge_sort.c

97 lines · C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define MAX 500000
5  void merge(int a[], int low, int mid, int high)
6  {
7      int i = low, j = mid + 1, k = 0;
8      int c[MAX];
9      while (i <= mid && j <= high)
10     {
11         if (a[i] <= a[j])
12             c[k++] = a[i++];
13         else
14             c[k++] = a[j++];
15     }
16     while (i <= mid)
17         c[k++] = a[i++];
18     while (j <= high)
19         c[k++] = a[j++];
20     for (i = low, j = 0; i <= high; i++, j++)
21         a[i] = c[j];
22 }
23 void mergeSort(int a[], int low, int high)
24 {
25     if (low >= high)
26         return;
27     int mid = (low + high) / 2;
28     mergeSort(a, low, mid);
29     mergeSort(a, mid + 1, high);
30     merge(a, low, mid, high);
31 }
32 int main()
33 {
34     int a[MAX];
35     int i, n, ch, value, p;
36     FILE *fp;
37     clock_t start, end;
38     double exetime;
39     while (1)
40     {
41         printf("\n===== MERGE SORT MENU =====\n");

```

```

42     printf("1. Merge Sort\n");
43     printf("2. Record Execution Time\n");
44     printf("3. Exit\n");
45     printf("Enter your choice: ");
46     scanf("%d", &ch);
47     switch (ch)
48     {
49         case 1:
50             printf("Enter number of elements: ");
51             scanf("%d", &n);
52             fp = fopen("input.txt", "w");
53             for (i = 0; i < n; i++)
54             {
55                 value = rand() % 10000;
56                 fprintf(fp, "%d\n", value);
57             }
58             fclose(fp);
59             printf("Random numbers stored in input.txt\n");
60             fp = fopen("input.txt", "r");
61             i = 0;
62             for (i = 0; i < n; i++)
63             {
64                 fscanf(fp, "%d", &a[i]);
65             }
66             fclose(fp);
67             mergeSort(a, 0, n - 1);
68             fp = fopen("output.txt", "w");
69             for (i = 0; i < n; i++)
70                 fprintf(fp, "%d\n", a[i]);
71             fclose(fp);
72             printf("Sorted elements stored in output.txt\n");
73             break;
74         case 2:
75             fp = fopen("plot.dat", "w");
76             for (p = 10000; p <= 100000; p += 10000)
77             {
78                 for (i = 0; i < p; i++)
79                     a[i] = rand() % 10000;
80                 start = clock();
81                 mergeSort(a, 0, p - 1);
82                 end = clock();
83                 exetime = (double)(end - start) / CLOCKS_PER_SEC;
84                 printf("N = %d\tTime = %lf seconds\n", p, exetime);
85                 fprintf(fp, "%d %lf\n", p, exetime);
86             }
87             fclose(fp);
88             printf("Execution times stored in plot.dat\n");
89             break;
90         case 3:
91             exit(0);
92         default:
93             printf("Invalid choice!\n");
94     }
95 }
96 return 0;
97 }

```

02

SORTING · DIVIDE & CONQUER

Quick Sort

TIME COMPLEXITY

$O(n \log n)$ average, $O(n^2)$ worst case

SPACE COMPLEXITY

$O(\log n)$ recursion stack (in-place)

STABLE

No

Key idea: Pick a **pivot** (here, the first element) and partition the array so everything smaller ends up on its left and everything larger on its right, then recursively sort each side. The partition step does the real work in-place using two scanning pointers, which is what makes Quick Sort fast in practice despite its $O(n^2)$ worst case on already-sorted or adversarial input.

02_quick_sort.c

112 lines · C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define MAX 500000
5  void swap(int *a, int *b)
6  {
7      int t = *a;
8      *a = *b;
9      *b = t;
10 }
11 int partition(int a[], int low, int high)
12 {
13     int pivot = a[low];
14     int i = low;
15     int j = high;
16     while (i < j)
17     {
18         while (i <= high - 1 && pivot >= a[i])
19             i++;
20         while (j >= low + 1 && pivot < a[j])
21             j--;
22         if (i < j)
23         {
24             swap(&a[i], &a[j]);
25         }
26     }
27     swap(&a[low], &a[j]);
28     return j;
29 }
30 void QuickSort(int a[], int low, int high)
31 {
32     if (low >= high)
33     {
34         return;
35     }
36     int s = partition(a, low, high);
37     QuickSort(a, low, s - 1);

```

```

42     QuickSort(a, s + 1, high);
43 }
44 int main()
45 {
46     int a[MAX];
47     int i, n, ch, value, p;
48     FILE *fp;
49     clock_t start, end;
50     double exetime;
51     while (1)
52     {
53         printf("\n===== QUICK SORT MENU =====\n");
54         printf("1. Quick Sort\n");
55         printf("2. Record Execution Time\n");
56         printf("3. Exit\n");
57         printf("Enter your choice: ");
58         scanf("%d", &ch);
59         switch (ch)
60         {
61             case 1:
62                 printf("Enter number of elements: ");
63                 scanf("%d", &n);
64                 fp = fopen("input.txt", "w");
65                 for (i = 0; i < n; i++)
66                 {
67                     value = rand() % 10000;
68                     fprintf(fp, "%d\n", value);
69                 }
70                 fclose(fp);
71                 printf("Random numbers stored in input.txt\n");
72                 fp = fopen("input.txt", "r");
73                 for (i = 0; i < n; i++)
74                 {
75                     fscanf(fp, "%d", &a[i]);
76                 }
77                 fclose(fp);
78                 QuickSort(a, 0, n - 1);
79                 fp = fopen("output.txt", "w");
80                 for (i = 0; i < n; i++)
81                 {
82                     fprintf(fp, "%d\n", a[i]);
83                 }
84                 fclose(fp);
85                 printf("Sorted elements stored in output.txt\n");
86                 break;
87             case 2:
88                 fp = fopen("plot.dat", "w");
89                 for (p = 10000; p <= 100000; p += 10000)
90                 {
91                     for (i = 0; i < p; i++)
92                     {
93                         a[i] = rand() % 10000;
94                     }
95                     start = clock();
96                     QuickSort(a, 0, p - 1);
97                     end = clock();
98                     exetime = (double)(end - start) / CLOCKS_PER_SEC;
99                     printf("N = %d\tTime = %lf seconds\n", p, exetime);
100                     fprintf(fp, "%d %lf\n", p, exetime);
101                 }
102                 fclose(fp);

```

```
103         printf("Execution times stored in plot.dat\n");
104         break;
105     case 3:
106         exit(0);
107     default:
108         printf("Invalid Choice!\n");
109     }
110 }
111 return 0;
112 }
```

03

GRAPH TRAVERSAL · QUEUE-BASED

Breadth First Search

TIME COMPLEXITY

 $O(V^2)$ for an adjacency matrix

SPACE COMPLEXITY

 $O(V)$ for the queue and visited array

EXPLORES

Level by level

Key idea: Starting from a source node, visit all of its direct neighbours first, then all of *their* unvisited neighbours, and so on — expanding outward one layer at a time. A simple array-based **queue** (front/rear indices) keeps track of which node to explore next, guaranteeing nodes are reported in increasing distance from the start.

03_bfs.c

51 lines · C

```

1  #include <stdio.h>
2  #define MAX 100 // Maximum number of vertices
3  void bfs(int graph[MAX][MAX], int visited[MAX], int n, int start)
4  {
5      int q[MAX]; // Simulated queue using an array
6      int f = 0, r = -1; // Indices for front and rear of the queue
7      visited[start] = 1;
8      q[++r] = start; // Enqueue the starting node
9      while (f <= r)
10     {
11         int cur = q[f++];
12         printf("%d ", cur);
13         for (int i = 0; i < n; ++i)
14         {
15             if (graph[cur][i] && !visited[i])
16             {
17                 visited[i] = 1;
18                 q[++r] = i; // Enqueue adjacent nodes
19             }
20         }
21     }
22     // Print nodes that are not reachable
23     printf("\nNodes not reachable from node %d: ", start);
24     for (int i = 0; i < n; ++i)
25     {
26         if (!visited[i])
27         {
28             printf("%d ", i);
29         }
30     }
31 }
32 int main()
33 {
34     int n, s, graph[MAX][MAX], visited[MAX] = {0};
35     printf("Enter number of vertices: ");
36     scanf("%d", &n);
37     printf("Enter adjacency matrix:\n");
38     for (int i = 0; i < n; i++)
39     {
40         for (int j = 0; j < n; j++)
41         {

```

```
42         scanf("%d", &graph[i][j]);
43     }
44 }
45 printf("Enter starting node: ");
46 scanf("%d", &s);
47 printf("Nodes reachable from node %d in BFS order:\n", s);
48 bfs(graph, visited, n, s);
49 printf("\n");
50 return 0;
51 }
```

04

GRAPH TRAVERSAL · RECURSIVE

Depth First Search

TIME COMPLEXITY

 $O(V^2)$ for an adjacency matrix

SPACE COMPLEXITY

 $O(V)$ recursion stack and visited array

EXPLORES

Branch by branch

Key idea: Starting from a source node, plunge as deep as possible along one path before backtracking — visit a node, then **recursively** visit the first unvisited neighbour, and only return to try the next neighbour once that entire branch is exhausted. The call stack implicitly remembers the path back.

04_dfs.c

34 lines · C

```

1  #include <stdio.h>
2  #define MAX 100 // Maximum number of vertices
3  void dfs(int graph[MAX][MAX], int visited[MAX], int n, int node)
4  {
5      printf("%d ", node);
6      visited[node] = 1;
7      for (int i = 0; i < n; i++)
8      {
9          if (graph[node][i] && !visited[i])
10             dfs(graph, visited, n, i);
11     }
12 }
13 int main()
14 {
15     int n, start, graph[MAX][MAX], visited[MAX] = {0};
16     printf("Enter number of vertices: ");
17     scanf("%d", &n);
18     printf("Enter adjacency matrix:\n");
19     for (int i = 0; i < n; i++)
20         for (int j = 0; j < n; j++)
21             scanf("%d", &graph[i][j]);
22     printf("Enter starting node: ");
23     scanf("%d", &start);
24     printf("Nodes reachable from node %d in DFS order:\n", start);
25     dfs(graph, visited, n, start);
26     printf("\n nodes that are not reachable from %d:\n", start);
27     for (int i = 0; i < n; i++)
28     {
29         if (!visited[i])
30             printf("%d ", i);
31     }
32     printf("\n");
33     return 0;
34 }
```

05

GRAPH ORDERING · KAHN'S ALGORITHM

Topological Sort

TIME COMPLEXITY

 $O(V^2)$ for an adjacency matrix

SPACE COMPLEXITY

 $O(V)$ for in-degree array and stack

REQUIRES

A directed acyclic graph (DAG)

Key idea: Repeatedly pick a vertex with **in-degree zero** (no remaining prerequisites), output it, and remove its outgoing edges — which may free up new zero-in-degree vertices. This is Kahn's algorithm: the order produced respects every directed edge $u \rightarrow v$ by always placing u before v .

05_topological_sort.c

58 lines · C

```

1  #include <stdio.h>
2  void topological(int a[10][10], int n)
3  {
4      int in[10], out[10], stack[10], top = -1;
5      int i, j, k = 0;
6      for (i = 0; i < n; i++)
7      {
8          in[i] = 0;
9          for (j = 0; j < n; j++)
10         {
11             if (a[j][i] == 1)
12                 in[i]++;
13         }
14     }
15     while (1)
16     {
17         for (i = 0; i < n; i++)
18         {
19             if (in[i] == 0)
20             {
21                 stack[++top] = i;
22                 in[i] = -1;
23             }
24         }
25         if (top == -1)
26             break;
27         out[k] = stack[top--];
28
29         for (i = 0; i < n; i++)
30         {
31             if (a[out[k]][i] == 1)
32                 in[i]--;
33         }
34         k++;
35     }
36     printf("\nTopological Sorting is:\n");
37     for (i = 0; i < k; i++)
38     {
39         printf("%d ", out[i] + 1);
40     }
41 }
42 int main()

```

```
43 {
44     int a[10][10], n, i, j;
45     printf("Enter the number of vertices: ");
46     scanf("%d", &n);
47     printf("Enter the adjacency matrix:\n");
48     for (i = 0; i < n; i++)
49     {
50         printf("Enter row %d:\n", i + 1);
51         for (j = 0; j < n; j++)
52         {
53             scanf("%d", &a[i][j]);
54         }
55     }
56     topological(a, n);
57     return 0;
58 }
```

06

SORTING · BINARY HEAP

Heap Sort

TIME COMPLEXITY

$O(n \log n)$ — best, average and worst case

SPACE COMPLEXITY

$O(1)$ extra — sorts in-place

STABLE

No

Key idea: First arrange the array into a **max-heap** so the largest element sits at index 0. Then repeatedly swap that root with the last unsorted element and **heapify** the shrinking heap to restore the max-heap property — each swap permanently places one more element in its final sorted position.

06_heap_sort.c

113 lines · C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define MAX 500000
5  void swap(int *a, int *b)
6  {
7      int t = *a;
8      *a = *b;
9      *b = t;
10 }
11 void heapify(int a[], int n, int i)
12 {
13     int largest = i;
14     int left = 2 * i + 1;
15     int right = 2 * i + 2;
16     if (left < n && a[left] > a[largest])
17     {
18         largest = left;
19     }
20     if (right < n && a[right] > a[largest])
21     {
22         largest = right;
23     }
24     if (largest != i)
25     {
26         swap(&a[i], &a[largest]);
27         heapify(a, n, largest);
28     }
29 }
30 void heapSort(int a[], int n)
31 {
32     int i;
33     // Build Max Heap
34     for (i = n / 2 - 1; i >= 0; i--)
35     {
36         heapify(a, n, i);
37     }
38     // Heap Sort
39     for (i = n - 1; i >= 1; i--)
40     {
41         swap(&a[0], &a[i]);
42         heapify(a, i, 0);

```

```

43     }
44 }
45 int main()
46 {
47     int a[MAX];
48     int i, n, ch, value, p;
49     FILE *fp;
50     clock_t start, end;
51     double exetime;
52     while (1)
53     {
54         printf("\n===== HEAP SORT MENU =====\n");
55         printf("1. Heap Sort\n");
56         printf("2. Record Execution Time\n");
57         printf("3. Exit\n");
58         printf("Enter your choice: ");
59         scanf("%d", &ch);
60         switch (ch)
61         {
62             case 1:
63                 printf("Enter number of elements: ");
64                 scanf("%d", &n);
65                 fp = fopen("input.txt", "w");
66                 for (i = 0; i < n; i++)
67                 {
68                     value = rand() % 10000;
69                     fprintf(fp, "%d\n", value);
70                 }
71                 fclose(fp);
72                 printf("Random numbers stored in input.txt\n");
73                 fp = fopen("input.txt", "r");
74                 for (i = 0; i < n; i++)
75                 {
76                     fscanf(fp, "%d", &a[i]);
77                 }
78                 fclose(fp);
79                 heapSort(a, n);
80                 fp = fopen("output.txt", "w");
81                 for (i = 0; i < n; i++)
82                 {
83                     fprintf(fp, "%d\n", a[i]);
84                 }
85                 fclose(fp);
86                 printf("Sorted elements stored in output.txt\n");
87                 break;
88             case 2:
89                 fp = fopen("plot.dat", "w");
90                 for (p = 10000; p <= 100000; p += 10000)
91                 {
92                     for (i = 0; i < p; i++)
93                     {
94                         a[i] = rand() % 10000;
95                     }
96                     start = clock();
97                     heapSort(a, p);
98                     end = clock();
99                     exetime = (double)(end - start) / CLOCKS_PER_SEC;
100                    printf("N = %d\tTime = %lf seconds\n", p, exetime);
101                    fprintf(fp, "%d %lf\n", p, exetime);
102                }
103                fclose(fp);

```

```
104         printf("Execution times stored in plot.dat\n");
105         break;
106     case 3:
107         exit(0);
108     default:
109         printf("Invalid Choice!\n");
110     }
111 }
112 return 0;
113 }
```

07

SHORTEST PATH · DYNAMIC PROGRAMMING

Floyd–Warshall Algorithm

TIME COMPLEXITY

 $O(V^3)$

SPACE COMPLEXITY

 $O(V^2)$ for the distance matrix

FINDS

All-pairs shortest distances

Key idea: For every intermediate vertex k , ask whether routing a path through k makes it shorter: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Doing this for every pair (i, j) and every possible intermediate k gradually relaxes every distance down to its true shortest-path value across the whole graph.

07_floyds.c

39 lines · C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  int MIN(int v1, int v2)
4  {
5      if (v1 >= v2)
6          return v2;
7      else
8          return v1;
9  }
10 int main()
11 {
12     int W[10][10], D[10][10], n, i, j, k;
13     printf("Enter the numbner of Vertices :");
14     scanf("%d", &n);
15     printf("Enter the Weight matrix of the given graph row wise \n");
16     for (i = 0; i < n; i++)
17         for (j = 0; j < n; j++)
18         {
19             scanf("%d", &W[i][j]);
20             D[i][j] = W[i][j];
21         }
22     for (k = 0; k < n; k++)
23     {
24         for (i = 0; i < n; i++)
25         {
26             for (j = 0; j < n; j++)
27                 D[i][j] = MIN(D[i][j], D[i][k] + D[k][j]);
28         }
29     }
30     printf("Solution to All pairs shortest distance problem\n");
31     printf("Using Floyd's Algorithm - Distance Matrix of the given graph is \n");
32     for (i = 0; i < n; i++)
33     {
34         for (j = 0; j < n; j++)
35             printf("%d\t", D[i][j]);
36         printf("\n");
37     }
38     return 0;
39 }
```

08

SHORTEST PATH · GREEDY

Dijkstra's Algorithm

TIME COMPLEXITY

 $O(V^2)$ for an adjacency matrix

SPACE COMPLEXITY

 $O(V)$ for distance and visited arrays

REQUIRES

Non-negative edge weights

Key idea: Maintain a tentative distance to every vertex, starting at 0 for the source and infinity elsewhere. Repeatedly pick the closest **unvisited** vertex, lock in its distance, and **relax** its neighbours' distances through it. Because weights are non-negative, once a vertex is visited its shortest distance can never improve.

08_dijkstra.c

71 lines · C

```

1  #include <stdio.h>
2  #define INFINITY 9999
3  #define MAX 10
4  void Dijkstra(int Graph[MAX][MAX], int n, int start)
5  {
6      int cost[MAX][MAX], distance[MAX], visited[MAX];
7      int count, mindistance, nextnode, i, j;
8      for (i = 0; i < n; i++)
9      {
10         for (j = 0; j < n; j++)
11         {
12             if (Graph[i][j] == 0)
13                 cost[i][j] = INFINITY;
14             else
15                 cost[i][j] = Graph[i][j];
16         }
17     }
18     for (i = 0; i < n; i++)
19     {
20         distance[i] = cost[start][i];
21         visited[i] = 0;
22     }
23     distance[start] = 0;
24     visited[start] = 1;
25     count = 1;
26     while (count < n - 1)
27     {
28         mindistance = INFINITY;
29         for (i = 0; i < n; i++)
30         {
31             if (distance[i] < mindistance && !visited[i])
32             {
33                 mindistance = distance[i];
34                 nextnode = i;
35             }
36         }
37         visited[nextnode] = 1;
38         for (i = 0; i < n; i++)
39         {
40             if (!visited[i] &&
41                 mindistance + cost[nextnode][i] < distance[i])
42                 {

```

```

43         distance[i] = mindistance + cost[nextnode][i];
44     }
45 }
46     count++;
47 }
48     printf("\nShortest Distances:\n");
49     for (i = 0; i < n; i++)
50     {
51         if (i != start)
52             printf("0 -> %d = %d\n", i, distance[i]);
53     }
54 }
55 int main()
56 {
57     int Graph[MAX][MAX];
58     int n, i, j;
59     printf("Enter number of vertices: ");
60     scanf("%d", &n);
61     printf("Enter adjacency matrix:\n");
62     for (i = 0; i < n; i++)
63     {
64         for (j = 0; j < n; j++)
65         {
66             scanf("%d", &Graph[i][j]);
67         }
68     }
69     Dijkstra(Graph, n, 0);
70     return 0;
71 }

```

09

MINIMUM SPANNING TREE · GREEDY

Kruskal's Algorithm

TIME COMPLEXITY

$O(V^2)$ here (scans the cost matrix each step)

SPACE COMPLEXITY

$O(V)$ for the union-find parent array

BUILDS THE MST

Edge by edge, cheapest first

Key idea: Sort — or in this matrix version, repeatedly scan for — the cheapest remaining edge, and add it to the spanning tree only if its two endpoints aren't already connected. A simple **union-find** (find / uni) tracks connected components so cycle-forming edges are skipped.

09_kruskals.c

62 lines · C

```

1  #include <stdio.h>
2  int i, j, a, b, u, v, n, ne = 1, min, Mcost = 0;
3  int cost[10][10], parent[10];
4  int find(int i)
5  {
6      while (parent[i])
7          i = parent[i];
8      return i;
9  }
10 int uni(int i, int j)
11 {
12     if (i != j)
13     {
14         parent[j] = i;
15         return 1;
16     }
17     return 0;
18 }
19 int main()
20 {
21     printf("\n\nImplementation of Kruskal's Algorithm\n\n");
22     printf("Input the number of vertices: ");
23     scanf("%d", &n);
24     printf("Input the adjacency matrix for the graph:\n");
25     for (i = 0; i < n; i++)
26     {
27         for (j = 0; j < n; j++)
28         {
29             scanf("%d", &cost[i][j]);
30             if (cost[i][j] == 0)
31                 cost[i][j] = 999;
32         }
33     }
34     printf("\nThe edges of MST are:\n");
35     printf("\nEdge\tWeight\n");
36     while (ne < n)
37     {
38         min = 999;
39         for (i = 0; i < n; i++)
40         {
41             for (j = 0; j < n; j++)
42                 if (cost[i][j] < min)

```

```
43         if (cost[i][j] < min)
44         {
45             min = cost[i][j];
46             a = u = i;
47             b = v = j;
48         }
49     }
50 }
51 u = find(u);
52 v = find(v);
53 if (uni(u, v))
54 {
55     printf("%d edge (%d - %d) = %d\n", ne++, a, b, min);
56     Mcost += min;
57 }
58 cost[a][b] = cost[b][a] = 999;
59 }
60 printf("\nCost of the Minimum Spanning Tree (MST) = %d\n", Mcost);
61 return 0;
62 }
```

10

MINIMUM SPANNING TREE · GREEDY

Prim's Algorithm

TIME COMPLEXITY

 $O(V^2)$ for an adjacency matrix

SPACE COMPLEXITY

 $O(V)$ for key, parent and mstSet arrays

BUILDS THE MST

Growing one tree outward

Key idea: Start the spanning tree from any single vertex, then repeatedly attach the **cheapest edge** that connects a vertex already in the tree to one that isn't. Unlike Kruskal's, the tree always stays a single connected piece, growing outward one edge at a time.

10_prims.c

71 lines · C

```

1  #include <stdio.h>
2  #define MAX 20
3  int minKey(int key[], int mstSet[], int n)
4  {
5      int min = 999, minIndex;
6      for (int i = 0; i < n; i++)
7      {
8          if (!mstSet[i] && key[i] < min)
9          {
10             min = key[i];
11             minIndex = i;
12         }
13     }
14     return minIndex;
15 }
16 void printMST(int parent[], int graph[MAX][MAX], int n)
17 {
18     int cost = 0;
19     printf("\nEdge\tWeight\n");
20     for (int i = 1; i < n; i++)
21     {
22         printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
23         cost += graph[i][parent[i]];
24     }
25     printf("Cost of MST = %d\n", cost);
26 }
27 void primMST(int graph[MAX][MAX], int n)
28 {
29     int parent[MAX];
30     int key[MAX];
31     int mstSet[MAX];
32     for (int i = 0; i < n; i++)
33     {
34         key[i] = 999;
35         mstSet[i] = 0;
36     }
37     key[0] = 0;
38     parent[0] = -1;
39     for (int count = 0; count < n - 1; count++)
40     {
41         int u = minKey(key, mstSet, n);
42         mstSet[u] = 1;

```

```
43     for (int v = 0; v < n; v++)
44     {
45         if (graph[u][v] && !mstSet[v] &&
46             graph[u][v] < key[v])
47         {
48             parent[v] = u;
49             key[v] = graph[u][v];
50         }
51     }
52 }
53 printMST(parent, graph, n);
54 }
55 int main()
56 {
57     int n;
58     int graph[MAX][MAX];
59     printf("Input the number of vertices: ");
60     scanf("%d", &n);
61     printf("Input the adjacency matrix:\n");
62     for (int i = 0; i < n; i++)
63     {
64         for (int j = 0; j < n; j++)
65         {
66             scanf("%d", &graph[i][j]);
67         }
68     }
69     primMST(graph, n);
70     return 0;
71 }
```

11

DYNAMIC PROGRAMMING · TABULATION

0/1 Knapsack

TIME COMPLEXITY

 $O(n \cdot W)$

SPACE COMPLEXITY

 $O(n \cdot W)$ for the DP table

VARIANT

0/1 — each item used at most once

Key idea: Build a table $V[i][j]$ holding the best profit using the first i items with capacity j . For each item, either **leave it out** (reuse the row above) or **take it** (add its profit to the best solution for the remaining capacity) — whichever value is larger. Walking the filled table backwards recovers which items were chosen.

11_knapsack.c

59 lines · C

```

1  #include <stdio.h>
2  int n, W;
3  int w[10], v[10], x[10], V[10][10];
4  int MAX(int a, int b)
5  {
6      return (a > b) ? a : b;
7  }
8  void Knapsack()
9  {
10     int i, j;
11     for (i = 0; i <= n; i++)
12     {
13         for (j = 0; j <= W; j++)
14         {
15             if (i == 0 || j == 0)
16                 V[i][j] = 0;
17             else if (j < w[i])
18                 V[i][j] = V[i - 1][j];
19             else
20                 V[i][j] = MAX(V[i - 1][j], V[i - 1][j - w[i]] + v[i]);
21             printf("%3d", V[i][j]);
22         }
23         printf("\n");
24     }
25 }
26 void PrintSolution()
27 {
28     int i = n, j = W;
29     while (i > 0 && j > 0)
30     {
31         if (V[i][j] != V[i - 1][j])
32         {
33             x[i] = 1;
34             j -= w[i];
35         }
36         i--;
37     }
38 }
39 int main()
40 {
41     int i;
42     printf("Enter number of objects: ");

```

```
43     scanf("%d", &n);
44     printf("Enter knapsack capacity: ");
45     scanf("%d", &W);
46     printf("Enter weight and profit:\n");
47     for (i = 1; i <= n; i++)
48         scanf("%d%d", &w[i], &v[i]);
49     printf("\nSolution Matrix:\n");
50     Knapsack();
51     PrintSolution();
52     printf("\nSelected Objects:\n");
53     printf("Object\tWeight\tProfit\n");
54     for (i = 1; i <= n; i++)
55         if (x[i])
56             printf("%d\t%d\t%d\n", i, w[i], v[i]);
57     printf("\nMaximum Profit = %d\n", V[n][W]);
58     return 0;
59 }
```

12

BACKTRACKING · CONSTRAINT SEARCH

N-Queens

TIME COMPLEXITY

 $O(N!)$ worst case

SPACE COMPLEXITY

 $O(N)$ for the board / column array

GOAL

Place N queens, no two attacking

Key idea: Place queens one row at a time. Before placing a queen in a column, **check** that no earlier queen shares that column or either diagonal. If a row has no safe column left, **backtrack** to the previous row and try its next option — exploring the search tree only as deep as constraints allow.

12_nqueens.c

76 lines · C

```

1  #include <math.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int board[20], count;
6
7  int main()
8  {
9      int n, i, j;
10     void queen(int row, int n);
11
12     printf(" - N Queens Problem Using Backtracking -");
13     printf("\n\nEnter number of Queens:");
14     scanf("%d", &n);
15     if (n == 2 || n == 3)
16         printf("\nSolution doesnot exists \n");
17     queen(1, n);
18     return 0;
19 }
20
21 // function for printing the solution
22 void print(int n)
23 {
24     int i, j;
25     printf("\n\nSolution %d:\n\n", ++count);
26
27     for (i = 1; i <= n; ++i)
28         printf("\t%d", i);
29
30     for (i = 1; i <= n; ++i)
31     {
32         printf("\n\n%d", i);
33         for (j = 1; j <= n; ++j) // for nxn board
34         {
35             if (board[i] == j)
36                 printf("\tQ"); // queen at i,j position
37             else
38                 printf("\t-"); // empty slot
39         }
40     }
41     printf("\n");
42 }

```

```

43
44 /*function to check conflicts
45 If no conflict for desired postion returns 1 otherwise returns 0*/
46 int place(int row, int column)
47 {
48     int i;
49     for (i = 1; i <= row - 1; ++i)
50     {
51         // checking column and digonal conflicts
52         if (board[i] == column)
53             return 0;
54         else if (abs(board[i] - column) == abs(i - row))
55             return 0;
56     }
57
58     return 1; // no conflicts
59 }
60
61 // function to check for proper positioning of queen
62 void queen(int row, int n)
63 {
64     int column;
65     for (column = 1; column <= n; ++column)
66     {
67         if (place(row, column))
68         {
69             board[row] = column; // no conflicts so place queen
70             if (row == n) // dead end
71                 print(n); // printing the board configuration
72             else // try queen with next position
73                 queen(row + 1, n);
74         }
75     }
76 }

```

13

STRING MATCHING · BAD-CHARACTER SHIFT

Horspool's Algorithm

TIME COMPLEXITY

$O(n)$ average case, $O(n \cdot m)$
worst case

SPACE COMPLEXITY

$O(\sigma)$ for the shift table ($\sigma =$
alphabet size, 256 here)

SKIP RULE

Bad-character shift, not a
position-by-position scan

Key idea: Align the pattern against a window of the text and compare **right to left** starting from the last character. On a mismatch, don't just slide one position — look up a precomputed **shift table** for the text character currently under the pattern's last position, and jump the window ahead by that amount, skipping alignments that can't possibly match.

13_horspool.c

42 lines · C

```

1  #include <stdio.h>
2  #include <string.h>
3  #define MAX 256
4  void shiftTable(char p[], int t[])
5  {
6      int m = strlen(p);
7      for (int i = 0; i < MAX; i++)
8          t[i] = m;
9      for (int i = 0; i < m - 1; i++)
10         t[p[i]] = m - 1 - i;
11 }
12 int horspool(char s[], char p[], int t[])
13 {
14     int n = strlen(s), m = strlen(p);
15     int i = m - 1;
16     while (i < n)
17     {
18         int k = 0;
19         while (k < m && p[m - 1 - k] == s[i - k])
20             k++;
21         if (k == m)
22             return i - m + 1;
23         i += t[s[i]];
24     }
25     return -1;
26 }
27 int main()
28 {
29     char text[MAX], pattern[MAX];
30     int table[MAX], pos;
31     printf("Enter text: ");
32     scanf("%s", text);
33     printf("Enter pattern: ");
34     scanf("%s", pattern);
35     shiftTable(pattern, table);
36     pos = horspool(text, pattern, table);
37     if (pos == -1)
38         printf("Pattern not found");
39     else
40         printf("Pattern found at position %d", pos + 1);

```

```
41     return 0;  
42 }
```

14

BRANCH & BOUND · OPTIMAL ASSIGNMENT

Job Assignment Problem

TIME COMPLEXITY

$O(n!)$ worst case — pruned heavily in practice

SPACE COMPLEXITY

$O(n)$ for the assignment / visited arrays

GOAL

Assign each person one job at minimum total cost

Key idea: Assign people to jobs one at a time, and before recursing further, compute a **lower bound** on the best total cost reachable from this partial assignment (the cheapest remaining job for every person not yet assigned). If that bound is already worse than the best complete solution found so far, **prune** the branch immediately — this is what avoids exploring all $n!$ assignments.

14_assignment.c

73 lines · C

```

1  #include <stdio.h>
2  int cost[20][20];
3  int assigned[20] = {0};
4  int assign[20];    // Current assignment
5  int bestAssign[20]; // Best assignment
6  int n, minCost = 999;
7  void assignment(int person, int currCost)
8  {
9      int job, i, j;
10     int bound = currCost;
11     // Calculate Lower Bound
12     for (i = person; i < n; i++)
13     {
14         int min = 999;
15         for (j = 0; j < n; j++)
16         {
17             if (assigned[j] == 0 && cost[i][j] < min)
18             {
19                 min = cost[i][j];
20             }
21         }
22         bound += min;
23     }
24     // Branch and Bound
25     if (bound >= minCost)
26     {
27         return;
28     }
29     // All persons assigned
30     if (person == n)
31     {
32         minCost = currCost;
33         for (i = 0; i < n; i++)
34         {
35             bestAssign[i] = assign[i];
36         }
37         return;
38     }
39     // Try every unassigned job
40     for (job = 0; job < n; job++)
41     {

```

```
42     if (assigned[job] == 0)
43     {
44         assigned[job] = 1;
45         assign[person] = job;
46         assignment(person + 1,
47                     currCost + cost[person][job]);
48         assigned[job] = 0;
49     }
50 }
51 }
52 int main()
53 {
54     int i, j;
55     printf("Enter number of persons/jobs: ");
56     scanf("%d", &n);
57     printf("Enter cost matrix:\n");
58     for (i = 0; i < n; i++)
59     {
60         for (j = 0; j < n; j++)
61         {
62             scanf("%d", &cost[i][j]);
63         }
64     }
65     assignment(0, 0);
66     printf("\nMinimum Cost = %d\n", minCost);
67     printf("\nOptimal Assignment:\n");
68     for (i = 0; i < n; i++)
69     {
70         printf("Person %d --> Job %d\n", i, bestAssign[i]);
71     }
72     return 0;
73 }
```